PRÁCTICA 1 · RA1

De datos sucios a un dataset fiable

Limpieza, transformación y validación con Pandas

M4 · Sistemes de Big Data
RA1 · Aplicación de técnicas de integración, procesamiento y análisis de información

Guía rápida

Duración orientativa 6-8 horas	Herramientas Python 3 · Pandas · NumPy · PyArrow · Colab o Jupyter	Entrega Notebook · CSV · Parquet · Informe PDF · ZIP

RUTA DE TRABAJO

Inspección → Limpieza → Transformación → Validación → CSV/Parquet → Reflexión

CHECKLIST ANTES DE ENTREGAR

- [ ] El notebook se ejecuta completo desde el archivo original.
- [ ] Las decisiones de limpieza están justificadas.
- [ ] Las métricas comparan el dataset original y el limpio.
- [ ] El ZIP contiene todos los archivos solicitados.

Rubén Gasque Todolí

Mapa de la práctica

Localiza rápidamente cada fase; la numeración ya corresponde a esta versión compacta.

1. Introducción 3
2. Objetivos de aprendizaje 3
3. Criterios de evaluación relacionados 3
4. Descripción del dataset 4
5. Parte 1: Inspección inicial 4
   5.1. Informe de problemas 5
6. Parte 2: Limpieza y transformación 5
   6.1. Variables textuales 5
   6.2. Tipos de datos 5
   6.3. Fechas 5
   6.4. Valores ausentes 6
   6.5. Duplicados 6
   6.6. Valores inválidos 6
7. Parte 3: Construcción del dataset final 7
8. Parte 4: Validación 7
   8.1. Comprobaciones automáticas 7
   8.2. Métricas de calidad 7
9. Parte 5: CSV vs. Parquet 8
10. Parte 6: Reflexión final 8
11. Requisitos técnicos 8
12. Condiciones de reproducibilidad 9
13. Entrega 9
14. Evaluación 10
15. Ampliación opcional: Polars 10
16. Pregunta final 10

1. Introducción

En un proyecto real de datos, la información rara vez llega perfectamente preparada para ser analizada. Es habitual encontrar valores ausentes, registros duplicados, tipos de datos incorrectos, categorías escritas de distintas formas, fechas incompatibles o valores que no tienen sentido desde el punto de vista del problema.

Antes de realizar cualquier análisis, construir un cuadro de mando o entrenar un modelo, es necesario transformar estos datos brutos en un conjunto de datos coherente, reproducible y suficientemente fiable.

En esta práctica trabajarás con el archivo:

`openfood_dirty.csv`

El dataset está basado en una muestra de datos de productos alimentarios procedente de *Open Food Facts* y ha sido adaptado para esta práctica. El conjunto contiene diferentes problemas de calidad y errores introducidos de forma controlada.

Tu objetivo será construir un pipeline reproducible que transforme el dataset original en un conjunto de datos limpio y preparado para posteriores análisis.

Objetivo final
Construir mediante Python y Pandas el siguiente flujo:

Datos brutos → Inspección → Limpieza → Transformación → Validación → Dataset fiable

El resultado final deberá generarse automáticamente a partir del archivo original, sin modificaciones manuales del CSV.

2. Objetivos de aprendizaje

Al finalizar la práctica deberás ser capaz de:

- Inspeccionar sistemáticamente un conjunto de datos desconocido.
- Identificar problemas de calidad y representación.
- Detectar valores ausentes, duplicados y valores inconsistentes.
- Corregir tipos de datos incorrectos.
- Procesar y normalizar fechas.
- Normalizar categorías y variables textuales.
- Detectar valores que no cumplen unas reglas de validez.
- Tomar decisiones justificadas sobre cómo tratar los problemas encontrados.
- Construir un pipeline de limpieza reproducible utilizando Pandas.
- Validar que el dataset generado cumple los requisitos establecidos.
- Comparar algunas características de CSV y Apache Parquet.
- Cuantificar la mejora de la calidad del dataset.

3. Criterios de evaluación relacionados

Esta práctica trabaja principalmente los siguientes criterios del RA1:

- **CE 1.2.** Extrae de forma automática información y conocimiento a partir de grandes volúmenes de datos.
- **CE 1.4.** Construye conjuntos de datos complejos y relaciona correctamente su información.

- **CE 1.5.** Establece objetivos, prioridades, secuenciación y organización del proceso.
- **CE 1.7.** Determina criterios de calidad necesarios para la eficacia y eficiencia de una solución Big Data.

4. Descripción del dataset

El archivo contiene información de productos y variables como:

Variable	Descripción
product_id	Identificador del producto.
product_name	Nombre comercial del producto.
brand	Marca.
category	Categoría del producto.
country	País asociado al producto.
quantity_g	Cantidad del producto expresada en gramos.
energy_kcal_100g	Energía por 100 gramos.
fat_100g	Grasas por 100 gramos.
sugars_100g	Azúcares por 100 gramos.
proteins_100g	Proteínas por 100 gramos.
salt_100g	Sal por 100 gramos.
nutriscore_grade	Categoría Nutri-Score.
created_at	Fecha de creación del registro.
last_modified	Fecha de última modificación.

El dataset contiene deliberadamente diferentes problemas.

Importante:
No se proporcionará una lista exacta de todos los errores presentes en el dataset.
Parte de la práctica consiste precisamente en **descubrirlos mediante una exploración sistemática de los datos**.

5. Parte 1: Inspección inicial

Antes de modificar cualquier dato debes comprender qué has recibido.
Carga el archivo utilizando Pandas y realiza una inspección inicial.
Como mínimo deberás analizar:

- dimensiones del dataset;
- nombres de columnas;
- tipos de datos;
- primeras y últimas filas;
- estadísticas descriptivas;
- número de valores ausentes por variable;
- porcentaje de valores ausentes;
- número de valores únicos;
- posibles registros duplicados;
- distribución de las principales variables categóricas;
- rangos de las variables numéricas.

Puedes utilizar funciones como:

```
df.head()
df.tail()
df.shape
df.info()
df.describe()
df.dtypes
df.isna().sum()
df.nunique()
df.duplicated().sum()
```

Estas funciones son únicamente orientativas. Puedes utilizar cualquier procedimiento adicional que consideres necesario.

5.1. Informe de problemas

Antes de empezar la limpieza, crea una tabla resumen similar a la siguiente:

Variable	Problema detectado	Decisión propuesta
Ejemplo	Categorías escritas de distintas formas	Normalizar categorías
Ejemplo	Valores ausentes	Analizar causa antes de decidir

La tabla debe contener los problemas que hayas identificado realmente en el dataset.

6. Parte 2: Limpieza y transformación

Construye un pipeline de limpieza utilizando Python y Pandas.
Todas las modificaciones deberán realizarse mediante código.

6.1. Variables textuales

Analiza y normaliza las variables textuales.
Deberás considerar, entre otros posibles problemas:

- espacios al inicio o final;
- diferencias entre mayúsculas y minúsculas;
- categorías equivalentes escritas de diferentes maneras;
- cadenas vacías;
- símbolos o caracteres innecesarios.

No debes modificar automáticamente todos los textos sin analizar previamente su significado.

6.2. Tipos de datos

Comprueba que cada variable utiliza un tipo de dato adecuado.
Por ejemplo:

- identificadores como texto;
- variables cuantitativas como valores numéricos;
- fechas como `datetime`;
- categorías como texto o tipo categórico.

Si una columna contiene valores como:

`"250 g"`, `"500g"`, `"1 kg"`

deberás decidir cómo convertirlos a una representación común.

6.3. Fechas

Las fechas pueden aparecer utilizando formatos diferentes.
El resultado final deberá utilizar tipos temporales adecuados.
Analiza:

- formatos diferentes;
- fechas no interpretables;
- posibles fechas incoherentes;
- relación entre `created_at` y `last_modified`.

Por ejemplo, deberás plantearte si sería coherente encontrar:

$$\texttt{last_modified} < \texttt{created_at}$$

6.4. Valores ausentes

Analiza individualmente las variables que contienen valores ausentes.
Para cada una de ellas deberás decidir si:

- eliminas el registro;
- mantienes el valor ausente;
- sustituyes el valor;
- recuperas la información utilizando otra variable;
- aplicas otra estrategia justificada.

> No se aceptará como estrategia general aplicar simplemente:
>
> ```
> df.dropna()
> ```
>
> sin analizar previamente qué datos se están eliminando y qué impacto tiene sobre el dataset.

6.5. Duplicados

Analiza la presencia de:

1. duplicados exactos;
2. identificadores de producto repetidos;
3. posibles duplicados lógicos.

Decide cómo tratar cada situación.
No todos los registros repetidos tienen necesariamente el mismo significado.

6.6. Valores inválidos

Define reglas de validez para las variables principales.
Como mínimo deberás comprobar:

- `quantity_g` >0;
- `energy_kcal_100g` dentro de un rango razonable;
- nutrientes expresados en gramos por 100 g dentro de rangos coherentes;
- `nutriscore_grade` dentro de las categorías permitidas;
- identificadores no nulos;
- coherencia temporal entre fechas.

Puedes definir reglas adicionales si detectas otros problemas.

El objetivo no es eliminar automáticamente cualquier fila problemática.
Debes decidir qué errores pueden corregirse y qué registros deberían descartarse cuando no es posible recuperar información fiable.

7. Parte 3: Construcción del dataset final

Una vez completada la limpieza, genera un nuevo DataFrame denominado:

`df_clean`

El dataset final deberá cumplir, como mínimo, las siguientes condiciones:

- `product_id` no contiene valores ausentes;
- `product_id` identifica de manera única cada producto;
- variables numéricas almacenadas como valores numéricos;
- fechas almacenadas como variables temporales;
- categorías principales normalizadas;
- duplicados tratados;
- ausencia de valores imposibles según las reglas definidas;
- nombres de variables consistentes.

Debes mostrar claramente el esquema final del dataset.
Incluye:

```
df_clean.info()
df_clean.head()
df_clean.describe()
```

y cualquier comprobación adicional necesaria.

8. Parte 4: Validación

No basta con ejecutar el código sin errores.
Debes demostrar que el dataset resultante es mejor que el original.

8.1. Comprobaciones automáticas

Incluye comprobaciones que permitan verificar algunas propiedades del dataset.
Por ejemplo:

```
assert df_clean["product_id"].notna().all()
assert df_clean["product_id"].is_unique
```

Añade tus propias validaciones para comprobar las reglas definidas durante la práctica.
Se valorará especialmente que estas comprobaciones sean automáticas.

8.2. Métricas de calidad

Calcula, como mínimo, las siguientes métricas antes y después de la limpieza:

Completitud

$$\text{Completitud} = \frac{\text{valores presentes}}{\text{valores esperados}} \times 100$$

Unicidad

Define una métrica apropiada utilizando el identificador de producto.

Validez

Calcula el porcentaje de registros que cumplen las reglas de validez definidas.
Presenta una tabla similar a:

Dimensión	Dataset original	Dataset limpio
Completitud	XX.X %	XX.X %
Unicidad	XX.X %	XX.X %
Validez	XX.X %	XX.X %

Interpreta los resultados.

9. Parte 5: CSV vs. Parquet

Exporta el dataset final en los dos formatos siguientes:

`openfood_clean.csv`

`openfood_clean.parquet`

Compara:

- tamaño ocupado por cada archivo;
- tipos de datos al volver a cargar cada fichero;
- principales diferencias observadas.

Responde brevemente:

1. ¿Qué formato ocupa menos espacio?
2. ¿Qué ocurre con los tipos de datos?
3. ¿Qué ventajas presenta Parquet para análisis de datos?
4. ¿En qué situaciones seguirías utilizando CSV?

No es necesario realizar un benchmark exhaustivo de rendimiento.

10. Parte 6: Reflexión final

Incluye una conclusión breve respondiendo a las siguientes cuestiones:

1. ¿Cuáles eran los principales problemas del dataset original?
2. ¿Qué decisiones de limpieza fueron las más difíciles de tomar?
3. ¿Hubo datos que decidiste no modificar? ¿Por qué?
4. ¿Qué porcentaje de registros se perdió durante el proceso?
5. ¿Ha mejorado objetivamente la calidad del dataset?
6. ¿Por qué puede ser peligroso limpiar datos automáticamente sin comprender su significado?
7. ¿Consideras que el dataset final está preparado para utilizarse en un análisis o cuadro de mando? Justifica la respuesta.

11. Requisitos técnicos

La práctica deberá realizarse utilizando:

- Python 3;
- Pandas;

- NumPy cuando sea necesario;
- PyArrow para trabajar con Parquet;
- Google Colab o Jupyter Notebook.

Puedes instalar dependencias en Colab mediante:

```
!pip install pyarrow
```

12. Condiciones de reproducibilidad

El notebook deberá poder ejecutarse desde el principio hasta el final partiendo únicamente de:

`openfood_dirty.csv`

No se permite:

- modificar manualmente el archivo CSV mediante Excel u otra aplicación;
- corregir manualmente registros individuales fuera de Python;
- entregar un dataset limpio que no pueda reproducirse ejecutando el notebook;
- eliminar grandes cantidades de datos sin justificar la decisión.

Antes de entregar, reinicia el entorno de ejecución y utiliza:

Ejecutar todas las celdas

El notebook debe generar automáticamente todos los resultados y archivos finales.

13. Entrega

La entrega deberá contener:

1. **Notebook de Python**
   `Apellido_Nombre_P1.ipynb`
2. **Dataset limpio en CSV**
   `openfood_clean.csv`
3. **Dataset limpio en Parquet**
   `openfood_clean.parquet`
4. **Informe breve en PDF**
   Máximo recomendado: 3–4 páginas.
   Debe incluir:
   - principales problemas detectados;
   - decisiones adoptadas;
   - métricas de calidad antes y después;
   - comparación CSV vs. Parquet;
   - conclusiones.

Todos los archivos deberán entregarse en un único archivo comprimido:

`Apellido_Nombre_P1.zip`

Apartado	Peso
Inspección inicial e identificación de problemas	15 %
Limpieza y transformación de los datos	30 %
Justificación de las decisiones adoptadas	15 %
Validación y métricas de calidad	20 %
Reproducibilidad y calidad del código	10 %
CSV, Parquet y conclusiones	10 %
Total	**100 %**

14. Evaluación

La práctica se evaluará sobre 10 puntos.
Se valorará especialmente:

- la capacidad para detectar problemas no indicados explícitamente;
- la coherencia de las decisiones;
- evitar soluciones automáticas injustificadas;
- la claridad del código;
- la reproducibilidad completa del pipeline;
- la validación objetiva del resultado.

15. Ampliación opcional: Polars

Este apartado es opcional y no es necesario para completar correctamente la práctica.
Selecciona dos operaciones realizadas durante la limpieza y reprodúcelas utilizando Polars.
Por ejemplo:

- lectura del dataset;
- filtrado de registros;
- transformación de una columna;
- agregación;
- lectura o escritura de Parquet.

Compara brevemente:

- sintaxis de Pandas y Polars;
- facilidad de uso;
- posibles diferencias conceptuales;
- rendimiento observado si el tamaño del dataset permite apreciarlo.

No es necesario reimplementar todo el pipeline con Polars.

16. Pregunta final

Un pipeline ha terminado correctamente, no ha generado ninguna excepción y ha creado el archivo final.

¿Significa esto que los datos son correctos?

Responde y justifica tu respuesta utilizando lo aprendido durante la práctica.